An important part of this is keeping your test code simple. Complex, convoluted test code is hard to understand and maintain, and is more difficult for others (and yourself, later on!) to understand.

One very effective way to make your code easier to read and to understand is to use fluent coding practices.

10.5.1 Using fluent coding to improve readability

Fluent coding can help you write code that communicates your intent more effectively. We say that code is *fluent* when it reads more like a natural-language sentence than like something written for a compiler.

For example, suppose you need to create a flight for your tests. Using conventional code in Java or .NET, you might write something like this:

```
Flight lastPlaneOut = new Flight("FH-525","Hong Kong","Sydney")
```

Although it's globally clear what this code does, there's still room for doubt. For example, you could probably guess that the first parameter represents the flight number, but does the flight leave from Hong Kong or Sydney?

With a little more effort, you could design the testing API so that the tests might read more like this:

```
Flight lastPlaneOut = Flight.number("FH-525").from("Sydney")
                                             .to("Hong Kong");
```

The idea of fluent code isn't language-specific. Dynamic languages like Ruby and Groovy, for example, support relatively fluent constructions natively:

```
def lastPlaneOut = new Flight(from: "Sydney",
                              to:"Hong Kong",
                              number:"FH-525")
```

The benefits of fluent code also apply to assertions. A well-written assertion tells you at a glance what a test is trying to demonstrate. It should be simple and obvious. You shouldn't have to decipher conditional logic or sift through `for` loops to understand what the code is doing. In addition, any nontrivial logic in a test case increases the risk of the test itself being wrong.

In recent years there has been a rise in the popularity of tools and techniques that make it easier to write more fluent code, both for production code and for tests. In the testing space, in particular, many libraries support fluent assertions in different languages. There are two main flavors to fluent assertions. The first typically uses the word "assert," whereas the second uses terms like "should" or "expect."

The first approach comes from a more traditional unit-testing background and focuses on testing and verification. Indeed, you typically make an assertion about something that has already happened, or a result that has already been calculated. The second is more BDD-centric: the words "should" and "expect" describe what you think the application should do, regardless of what it does currently, or if it even exists.

Let's look at some examples of fluent assertion libraries in different languages.